CLA 4: ML Model Deployment Strategies and Model Serving using FastAPI

Dataset: Titanic — Machine Learning from Disaster (Kaggle)

Dataset link: https://www.kaggle.com/competitions/titanic/data

Objective

Train a classification model, save the complete preprocessing/model pipeline, demonstrate online, batch and streaming inference, and serve predictions through a FastAPI REST API.

Implementation summary

The solution uses the specified Kaggle train.csv file. It selects passenger class, sex, age, siblings/spouses aboard, parents/children aboard, fare and embarkation port as input features. Missing numeric values are filled using the median; missing categorical values are filled using the most frequent value. Categorical values are one-hot encoded and numeric features are scaled. Logistic Regression is trained on an 80% training split and evaluated on a 20% stratified test split. The complete pipeline is saved as model.pkl using Joblib, and the held-out row indices are saved for reproducible inference demonstrations.

Inference strategies

Online inference predicts one passenger input. Batch inference predicts 20 records from the held-out test split together and writes outputs/batch_predictions.csv. Streaming inference processes ten records from the held-out test split one at a time with a one-second delay. All strategies load the same model.pkl and do not retrain.

FastAPI model serving

GET /health reports API status and whether the model is loaded. POST /predict accepts passenger details, validates them with Pydantic, and returns the predicted class and a readable label. Swagger UI is available at /docs. A valid request should return a prediction; an invalid value such as Pclass=5 or Sex='other' should be rejected with HTTP 422. The Python client sends a valid request to the API.

Task 6 — Observations

1. Online vs Batch: Online inference predicts one record per request and is useful when an immediate result is needed. Batch inference predicts a group of records together and is useful for processing many records efficiently.

2. Streaming vs Batch: Streaming processes records one by one as they arrive. Batch inference processes a group together after the group is available. This assignment simulates streaming with a one-second delay.

3. Why FastAPI? FastAPI exposes the model through HTTP endpoints, validates request data with Pydantic, and provides interactive Swagger documentation at /docs.

4. Why reuse the same model? Reusing the saved pipeline keeps preprocessing and predictions consistent across inference modes and avoids unintended model-version differences or unnecessary retraining.

Results and evidence to add after running

Run the scripts with the official Kaggle train.csv. Record the measured test accuracy from train.py and capture screenshots of online inference, batch inference, streaming inference, successful /health and /predict requests, and an invalid request rejected with HTTP 422. Do not report an accuracy value or claim screenshots that have not been generated by an actual run.

Execution commands

```
pip install -r requirements.txt
python train.py
```

```
python online_inference.py
python batch_inference.py
python streaming_inference.py
uvicorn app:app --reload
```